STEP 07 OF 24

Create a project

Use django-admin to generate the project files, then start the development server and watch Django run for the first time.

IN THIS STEP

01 Understand what a project is

02 Run django-admin startproject

03 Read the command word by word

04 Start the development server

What is a Django project?

A project is the whole website: its settings, its main URL list and the apps that plug into it.

Project

student_management/

- The container for one whole website.
- Holds the settings, the main URL list and the server entry files.
- Made once with django-admin startproject.

App

students/ (Step 09)

- One feature of the website, such as students or courses.
- Holds its own models, views, templates and tests.
- Made many times with manage.py startapp.

Settings package

config/

- A small Python folder inside the project.
- Holds settings.py, urls.py, wsgi.py and asgi.py.
- Named config so it never clashes with the project folder.

Think of a project as a house and apps as its rooms: one house, many rooms, and each room has one job.

Name the project and mind the dot

- **Pick a simple name.** Use lowercase letters, numbers and underscores. A name like my-site fails because Python cannot import a dash.
- **Avoid reserved names.** Do not call the project django or test. They clash with real Python modules.
- **We use config.** It names the folder that holds settings.py, so it never matches the outer folder student_management.
- **The dot matters.** The . at the end means: use the current folder. Without it, Django makes one more folder and nests everything inside.
- **Check your place.** Run pwd first, to be sure you are inside student_management.

With the dot (use this)

```
student_management/
├── config/
├── manage.py
└── venv/
```

Without the dot (avoid this)

```
student_management/
└── config/
    ├── config/
    └── manage.py
```

Creating the project

1 `cd ~/projects/student_management`

Go to the project folder you made in Step 04.

2 `source venv/bin/activate`

Turn on the virtual environment. Look for (venv) at the start of the prompt.

3 `django-admin startproject config .`

Create the project here. Do not forget the dot at the end.

4 `ls`

List the folder to see what was created.

Result of ls

```
config  manage.py
requirements.txt  venv
```

Django prints nothing when startproject works. No message means success.

Two new things appeared: manage.py and the config folder. Step 08 explains every file inside them.

Reading the command word by word

The full command

```
(venv) $ django-admin startproject config .
```

- **Same tool, other way.** python -m django startproject config . does exactly the same job.
- **Other folder.** Add a path at the end, such as ~/sites/demo, to create the project somewhere else.
- **Safe to repeat?** Not quite. If manage.py already exists, Django stops with a CommandError and overwrites nothing.

Part	What it means
django-admin	The command-line tool installed with Django. It works anywhere inside your active virtual environment.
startproject	The sub-command that builds a new project skeleton: manage.py plus the settings package.
config	The name of the settings package. It becomes a Python folder with an __init__.py file inside.
.	The destination. A single dot means the folder you are standing in right now.

Starting the development server

1 `python manage.py runserver`

Start Django's built-in test server. Run it from the folder that contains manage.py.

2 `http://127.0.0.1:8000/`

Open this address in your browser. 127.0.0.1 means this computer and 8000 is the port.

3 `Ctrl + C`

Press these keys in the terminal to stop the server.

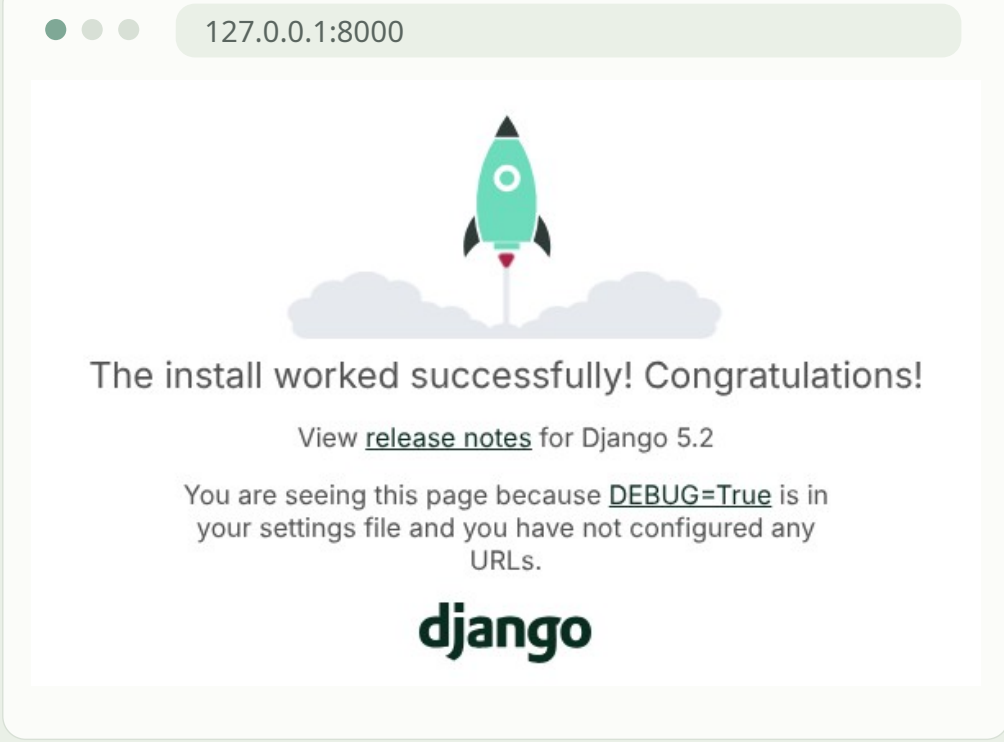
EXAMPLE OUTPUT (ABRIDGED)

```
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
You have 18 unapplied migration(s). ...
Run 'python manage.py migrate' to apply them.
Django version 5.2.x, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

⚠ runserver is only for testing. Never use it to host a real website.

The Django welcome page

- **It means success.** The rocket page proves that Python, your virtual environment and Django all work together.
- **Why you see it.** DEBUG is True in settings.py and you have no URLs of your own yet, so Django shows a friendly default page.
- **It is temporary.** After you add your own URLs in Step 11, this page is replaced by your pages.
- **Page does not open?** Check that the server is still running, that the address matches the terminal, and that you typed http:// and not https://.
- **Any browser works.** Firefox, Chrome and Edge all show the same page. It is served only to your own computer.



Useful runserver options

Command	What it does
python manage.py runserver	Start the server at 127.0.0.1:8000. This is the normal way.
python manage.py runserver 8080	Use another port, for example when 8000 is already busy.
python manage.py runserver 0.0.0.0:8000	Listen on every network address so a phone or another computer can open the site. Add your IP address to ALLOWED_HOSTS first.
python manage.py runserver --noreload	Turn off auto-reload. Normally the server restarts by itself every time you save a file.
python manage.py check	Look for problems in your project without starting the server.

⚠ runserver is slow and not secure by design, so it is for development only. Step 22 shows how to run a real production server.

Reading the server messages

Message in the terminal	What it means
Watching for file changes with StatReloader	Auto-reload is on. Save a file and the server restarts by itself.
System check identified no issues (0 silenced).	Django checked your project and found no problems.
You have 18 unapplied migration(s).	The built-in apps need database tables. We create them in Step 15, so ignore this for now.
Django version 5.2.x, using settings 'config.settings'	Shows which Django version is running and which settings file it reads.
Starting development server at http://127.0.0.1:8000/	The address to open in your browser.
"GET / HTTP/1.1" 200	One line per page request: method, path and status code. 200 means OK and 404 means not found.

Checkpoint and common errors

Before you continue

- ls shows config and manage.py
- runserver starts with no errors
- The rocket page opens
- Ctrl + C stops the server

Message	What to do
django-admin: command not found	Activate the venv with source venv/bin/activate.
CommandError: manage.py already exists	A project is already here. Run ls and do not run startproject twice.
Couldn't import Django	The venv is not active. Activate it and try again.
That port is already in use.	Stop the other server or use runserver 8080.

UP NEXT · STEP 08 **Project structure** manage.py, settings.py and more

STEP 08 OF 24

Project structure

Learn what every generated file does, and read the default code of each one line by line.

IN THIS STEP

01 Map of the project folders

02 manage.py and __init__.py

03 settings.py in detail

04 urls.py, wsgi.py and asgi.py

The project map

Project layout

```
student_management/          the project folder
├── venv/                  your virtual environment
├── requirements.txt         saved package list
├── manage.py              command-line helper
└── config/                the settings package
    ├── __init__.py          marks a Python package
    ├── settings.py          all project settings
    ├── urls.py            the main URL list
    ├── wsgi.py              entry point for WSGI servers
    └── asgi.py            entry point for ASGI servers
```

- **You made** student_management/, venv/ and requirements.txt in Steps 04 to 06.
- **Django made** manage.py and the config/ folder in Step 07.
- **Coming later** students/ in Step 09, templates/ in Step 13, db.sqlite3 in Step 15 and static/ in Step 20.
- **Ignore** __pycache__/ folders. Python creates them so files load faster.

How the files work together

Follow one command through the files.

1

You

terminal

Type python manage.py runserver.

2

manage.py

starts Django

Chooses config.settings and runs your command.

3

settings.py

configures

Tells Django about apps, database and middleware.

4

wsgi.py

server entry

Gives the web server the Django application object.

5

urls.py

routes

Sends each page request to the right view.

manage.py starts things, settings.py configures them, urls.py routes requests, and wsgi.py with asgi.py connect Django to web servers.

__init__.py: the empty file

- **It is empty.** The file has zero lines of code. That is exactly how Django ships it.
- **It marks a package.** A folder that holds __init__.py is a Python package, so Python can import config.settings and config.urls.
- **Why it matters.** Django finds your settings with the dotted name config.settings. That works only because config is a package.
- **Do not delete it.** Imports can break without it, even though the file looks useless.
- **About __pycache__.** Python adds folders named __pycache__ with compiled copies of your files. Ignore them and add them to .gitignore.

config/__init__.py

(empty file: 0 lines)

Imports that depend on it

```
from config import settings
import config.urls
```

manage.py: imports and main()

manage.py (part 1)

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""

import os
import sys


def main():
    """Run administrative tasks."""
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
```

- **#!/usr/bin/env python.** A shebang line. On Linux it lets the system find Python when the file runs directly.
- **Docstring.** Text in triple quotes that describes the file or function. Python ignores it when running.
- **import os, sys.** os talks to the operating system. sys gives the words you type after the command.
- **def main():** Puts the work inside a function, so nothing runs just because the file was imported.
- **setdefault.** Tells Django where your settings are: the module config.settings. If the variable already exists, it is left alone.

manage.py: running your command

manage.py (part 2)

```
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == "__main__":
    main()
```

- **try / except.** Django must be installed for this file to work. If Python cannot import it, the code jumps to the except part.
- **The friendly error.** raise ImportError(...) prints the long message you may have seen: check the install and activate the virtual environment.
- **execute_from_command_line.** Django's own function. It reads what you typed after manage.py and runs that command.
- **sys.argv.** The list of typed words, such as ['manage.py', 'runserver'].
- **if __name__ == '__main__'.** Run main() only when you start this file directly, not when another file imports it.

Commands you can run with manage.py

Command	What it does	Step
runserver	Start the development server.	07
startapp	Create a new app folder, such as students.	09
makemigrations	Write migration files from your models.	15
migrate	Apply migrations to the database.	15
shell	Open a Python shell with Django loaded.	16
createsuperuser	Create a login for the admin site.	18
test	Run the automated tests.	21
check	Look for problems in the project.	07
help	List every available command.	—

settings.py at a glance

One normal Python file. Each setting is a variable written in CAPITAL letters, in this order.

01 **Docstring and import**
Describes the file and loads Path

02 **BASE_DIR**
The path of the project folder

03 **SECRET_KEY**
A private key for signing data

04 **DEBUG**
Detailed errors in development

05 **ALLOWED_HOSTS**
Which host names may use the site

06 **INSTALLED_APPS**
The apps that are switched on

07 **MIDDLEWARE**
Helpers all requests pass through

08 **ROOT_URLCONF**
The file with the main URL list

09 **TEMPLATES**
How HTML templates are found

10 **WSGI_APPLICATION**
Entry point for WSGI web servers

11 **DATABASES**
Which database to use, and where

12 **AUTH_PASSWORD_VALIDATORS**
Rules for stronger passwords

13 **LANGUAGE_CODE, TIME_ZONE**
Language, clock, I18N and TZ flags

14 **STATIC_URL**
Web address of CSS, JS and images

15 **DEFAULT_AUTO_FIELD**
Type of the automatic id column

settings.py: header and BASE_DIR

config/settings.py

```
"""
Django settings for config project.

Generated by 'django-admin startproject' using Django 5.2.x.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""

from pathlib import Path

# Build paths inside the project like this:
# BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent
```

- **Docstring.** The text in triple quotes explains the file and records which Django version created it.
- **from pathlib import Path.** Path is Python's modern tool for file paths. It works the same on Linux, Windows and macOS.
- **__file__.** The full path of this file: .../student_management/config/settings.py.
- **resolve().** Turns that into a clean, absolute path.
- **parent.parent.** Each .parent goes up one folder: first to config/, then to student_management/. That folder is BASE_DIR.
- **Use it like this.** BASE_DIR / 'db.sqlite3' builds a path inside the project. The slash joins the parts.

settings.py: SECRET_KEY and DEBUG

- **SECRET_KEY.** A long random string. Django uses it to sign cookies, sessions and password-reset links, so nobody can fake them.
- **django-insecure-.** This start means Django made the key for development. Replace it before you go live.
- **Keep it private.** Never put a real key on GitHub. In Step 22 we read it from an environment variable.
- **DEBUG = True.** Shows a detailed error page with your code and settings, and reloads the server after each change.
- **Production rule.** Set DEBUG to False on a real server. Otherwise visitors could see your secrets in error pages.

config/settings.py

```
# SECURITY WARNING: keep the secret key
# used in production secret!
SECRET_KEY = "django-insecure-RANDOM_TEXT"

# SECURITY WARNING: don't run with debug
# turned on in production!
DEBUG = True
```

settings.py: ALLOWED_HOSTS

- **What it is.** A list of host names (website addresses) that your project will answer to.
- **Empty list, DEBUG True.** Django allows only localhost, 127.0.0.1 and [::1]. That is why runserver works with no setup.
- **Empty list, DEBUG False.** Django refuses every request with a 400 Bad Request error.
- **Real sites.** List your domain. A leading dot, as in .example.com, also allows its sub-domains.
- **Testing on a phone.** Add your computer's IP address, then run runserver 0.0.0.0:8000.
- **Avoid the star.** ["*"] allows any host. Use it only for quick tests, never in production.

config/settings.py

```
ALLOWED_HOSTS = []
```

Examples you may write later

```
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
ALLOWED_HOSTS = ["example.com", ".example.com"]
ALLOWED_HOSTS = ["*"]
```

settings.py: INSTALLED_APPS

config/settings.py

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]
```

App	What it gives you
admin	The admin site where you manage data (Step 18).
auth	Users, groups, permissions and login.
contenttypes	A record of every model. Auth and admin use it.
sessions	Remembers each visitor between page loads.
messages	One-time notes such as 'Student saved'.
staticfiles	Finds CSS, JavaScript and image files.



Your own apps go at the end of this list. In Step 09 we add "students" here.

settings.py: MIDDLEWARE

- **Middleware.** Small helpers between the web server and your views. Every request and every response passes through them.
- **Think airport checks.** Each desk does one job, one after the other.
- **Order matters.** A request goes down the list from top to bottom. The response comes back up from bottom to top.
- **What they can do.** Change the request, stop it early, or change the response.
- **Dotted paths.** Each line names a Python class. You can add your own or third-party ones to this list.

config/settings.py

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

What each middleware does

Middleware	What it does for you
SecurityMiddleware	Adds security headers and can send visitors from http to https.
SessionMiddleware	Loads the visitor's session on the way in and saves it on the way out.
CommonMiddleware	Does everyday URL fixes, such as adding a missing slash: /students becomes /students/.
CsrfViewMiddleware	Blocks forged form submissions from other websites (CSRF protection).
AuthenticationMiddleware	Adds request.user, so every view knows who is logged in.
MessageMiddleware	Supports one-time messages such as 'Student saved'.
XFrameOptionsMiddleware	Stops other sites from showing yours inside a frame (clickjacking protection).



Keep the order. AuthenticationMiddleware and MessageMiddleware both need SessionMiddleware above them.

settings.py: URL and WSGI settings

- **ROOT_URLCONF.** Tells Django which file holds the main URL list. config.urls means the file config/urls.py.
- **Dotted paths.** Python writes folders and files with dots instead of slashes, and leaves out .py.
- **WSGI_APPLICATION.** Points to the variable application inside config/wsgi.py. The development server and WSGI servers use it.
- **Renaming config?** If you ever rename the config folder, change both lines too.
- **No ASGI line.** The default file has no ASGI_APPLICATION setting. ASGI servers are pointed at config.asgi:application on the command line.

config/settings.py

```
ROOT_URLCONF = "config.urls"

WSGI_APPLICATION = "config.wsgi.application"
```

Dotted path to file

```
config.urls              → config/urls.py
config.wsgi.application  → config/wsgi.py
                           variable: application
```

settings.py: TEMPLATES

config/settings.py

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

- **BACKEND.** The template engine. Django's own engine is called DjangoTemplates.
- **DIRS.** Extra folders to search for templates. It is empty now. We add BASE_DIR / 'templates' in Step 13.
- **APP_DIRS.** True means Django also looks inside a templates/ folder in every installed app.
- **OPTIONS.** Extra engine settings. The main one is the list of context processors, explained next.

Context processors in TEMPLATES

- **What is it?** A small function that adds extra variables to every template automatically.
- **request.** Gives templates the current request, for example {{ request.path }}.
- **auth.** Adds user and perms, so any page can show {{ user.username }} or check permissions.
- **messages.** Adds messages, the one-time notes that MessageMiddleware stores.
- **Why useful.** You write the variable once in settings, not in every single view.

Used inside any template

```
<p>Hello, {{ user.username }}</p>
<p>You are on {{ request.path }}</p>

{% for message in messages %}
  <p>{{ message }}</p>
{% endfor %}
```

settings.py: DATABASES

- **A dictionary.** DATABASES holds one entry for every database your project uses.
- **default.** The main database. Django uses it unless you ask for another.
- **ENGINE.** Chooses the database software. SQLite is a single file, so it needs no server and no password.
- **NAME.** For SQLite, the path of the file. BASE_DIR / 'db.sqlite3' puts it in the project folder.
- **Created later.** The file db.sqlite3 appears when you run migrate in Step 15.
- **Real projects.** Larger sites switch to PostgreSQL or MySQL and add USER, PASSWORD, HOST and PORT.

config/settings.py

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Other engines you can use

```
django.db.backends.postgresql
django.db.backends.mysql
django.db.backends.oracle
```

settings.py: AUTH_PASSWORD_VALIDATORS

config/settings.py

```
AUTH_PASSWORD_VALIDATORS = [
    {
        "NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.CommonPasswordValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.NumericPasswordValidator",
    },
]
```

These rules run in forms, in the admin site and in createsuperuser.

UserAttributeSimilarity

Rejects a password that looks like the username, name or email.

MinimumLength

Needs at least 8 characters. You can change this with OPTIONS.

CommonPassword

Rejects well-known passwords such as password123.

NumericPassword

Rejects passwords made only of digits.

settings.py: language and time

- **LANGUAGE_CODE.** The language for Django's own messages, such as the admin site and error text. 'en-us' is American English.
- **TIME_ZONE.** The default clock for your site. UTC is the world standard. Use a name like 'Asia/Karachi' to match your city.
- **USE_I18N.** Turns on translation. I18N is short for internationalization: 18 letters between I and N.
- **USE_TZ.** True stores every date and time with its time zone, in UTC. This avoids mistakes when users live in different zones.
- **Safe to leave.** The defaults work. Changing TIME_ZONE is the one change most projects make.

config/settings.py

```
LANGUAGE_CODE = "en-us"

TIME_ZONE = "UTC"

USE_I18N = True

USE_TZ = True
```

Example: your local time zone

```
TIME_ZONE = "Asia/Karachi"
```

settings.py: static files and id type

- **STATIC_URL.** The web address prefix for CSS, JavaScript and images. Files are served under /static/.
- **Where files live.** Django looks in a static/ folder inside each app. In Step 20 we add our own folder too.
- **Production.** On a real server, collectstatic gathers all static files into one place (Step 22).
- **DEFAULT_AUTO_FIELD.** Sets the type of the automatic id column that every model gets.
- **BigAutoField.** A whole number that counts up 1, 2, 3 and so on, with room for billions of rows.
- **Same in apps.py.** startapp also writes this value into each app's apps.py, as you will see in Step 09.

config/settings.py

```
STATIC_URL = "static/"

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

How Django uses your settings

- **Plain Python.** settings.py is a normal Python file. Only names in CAPITAL letters count as settings.
- **How Django finds it.** manage.py sets DJANGO_SETTINGS_MODULE to config.settings before any command runs.
- **Reading settings.** In your own code, import settings from django.conf. Never import settings.py directly.
- **See your changes.** python manage.py diffsettings lists every setting that differs from Django's built-in defaults.
- **Add your own.** You can add new CAPITAL names, such as SCHOOL_NAME = 'Demo School', and read them the same way.
- **Check for mistakes.** python manage.py check reads your settings and reports problems.

Read a setting in your code

```
from django.conf import settings

print(settings.DEBUG)
print(settings.BASE_DIR)
print(settings.TIME_ZONE)
```

Output (shortened)

```
$ python manage.py diffsettings
DEBUG = True
ROOT_URLCONF = 'config.urls'
SECRET_KEY = 'django-insecure-...'
TIME_ZONE = 'UTC'
WSGI_APPLICATION = 'config.wsgi.application'
```

Settings you will add later

Setting	When	Why
INSTALLED_APPS	Step 09	Add "students" so Django knows about your app.
TEMPLATES → DIRS	Step 13	Add BASE_DIR / "templates" for project-wide HTML files.
STATICFILES_DIRS	Step 20	Tell Django about your own static folder.
LOGIN_URL, LOGIN_REDIRECT_URL	Step 19	Choose where login sends people.
STATIC_ROOT	Step 22	The folder where collectstatic gathers files.
SECRET_KEY, DEBUG, ALLOWED_HOSTS	Step 22	Read the real values from environment variables.
TIME_ZONE	Optional	Change UTC to your own time zone.



After every change to settings.py, save the file and watch the runserver terminal for errors.

urls.py: the default code

- **Docstring.** The long text at the top shows three ways to add a URL. The examples are in the panel on the right.
- **import admin.** Gives you the built-in admin site (Step 18).
- **import path.** path() builds one URL rule.
- **urlpatterns.** The list Django reads from top to bottom to match each request.
- **path('admin/', ...).** Any address that starts with admin/ goes to the admin site.
- **One route only.** Only one route exists, so /students/ gives a 404 error until Step 11.

config/urls.py

```
from django.contrib import admin
from django.urls import path

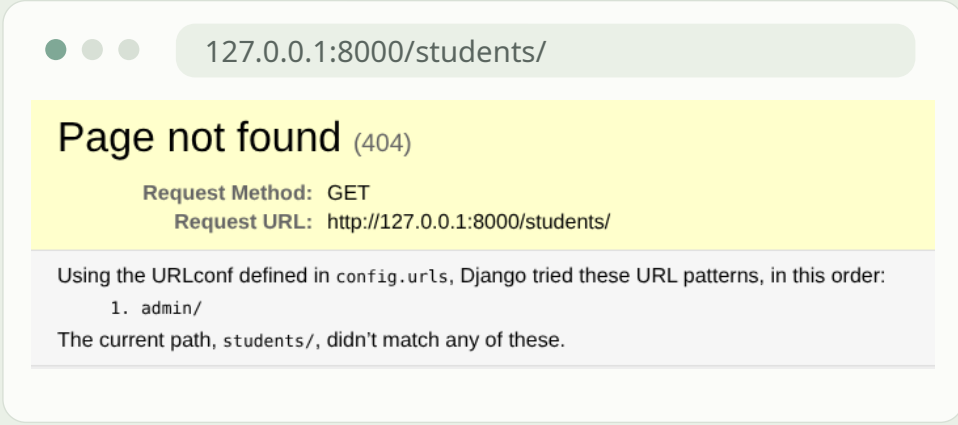
urlpatterns = [
    path("admin/", admin.site.urls),
]
```

The three examples in the docstring

```
path('', views.home, name='home')
path('', Home.as_view(), name='home')
path('blog/', include('blog.urls'))
```

How Django matches a URL

- **1. Take the path.** For http://127.0.0.1:8000/admin/login/ the path is admin/login/.
- **2. Compare in order.** Django checks each path() in urlpatterns, starting at the top.
- **3. First match wins.** admin/ matches, so the rest (login/) goes to the admin site.
- **4. No match, 404.** If nothing matches, Django shows Page not found.
- **Debug 404 page.** With DEBUG True, the 404 page lists every pattern Django tried. It is the fastest way to fix a wrong URL.
- **Try it yourself.** Open the three addresses on the right in your browser and compare what you see.



Try these addresses

/ welcome page
/admin/ redirects to the login
/students/ 404 page not found

wsgi.py: the default code

config/wsgi.py

```
"""
WSGI config for config project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/howto/deployment/wsgi/
"""

import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

application = get_wsgi_application()
```

- **WSGI.** Web Server Gateway Interface. A standard way for a Python web app to talk to a web server.
- **import os.** Used to set an environment variable.
- **setdefault.** Points Django to config.settings, just like manage.py does.
- **get_wsgi_application().** Builds the Django app that web servers will call.
- **application.** The name servers look for. WSGI_APPLICATION in settings.py points here.
- **When used.** runserver uses it, and so does Gunicorn in Step 22: gunicorn config.wsgi.

asgi.py: the default code

config/asgi.py

```
"""
ASGI config for config project.

It exposes the ASGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/howto/deployment/asgi/
"""

import os

from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

application = get_asgi_application()
```

- **ASGI.** Asynchronous Server Gateway Interface. A newer standard that also supports async code and WebSockets.
- **Same pattern.** The file looks like wsgi.py. Only the function changes to get_asgi_application().
- **application.** The variable that ASGI servers such as Uvicorn or Daphne import.
- **Not used yet.** runserver uses wsgi.py, so asgi.py waits until you deploy with an ASGI server.
- **Why keep it.** Django creates it so async features are ready when you need them.

WSGI vs ASGI at a glance

Aspect	WSGI (wsgi.py)	ASGI (asgi.py)
Full name	Web Server Gateway Interface	Asynchronous Server Gateway Interface
Age	Older and very stable	Newer standard
Handles	Normal request and response pages	Normal pages plus async views and WebSockets
Style	One request per worker at a time	Many connections can wait at once
Example servers	Gunicorn, uWSGI, mod_wsgi	Uvicorn, Daphne, Hypercorn
Start command	gunicorn config.wsgi	uvicorn config.asgi:application
Choose it when	You build normal pages and forms, like our project	You need real-time features such as chat



In development, runserver handles all of this for you. We choose a real server in Step 22.

Checkpoint: which file does what?

Before you continue

- I know what manage.py does
- I can find BASE_DIR in settings.py
- I can explain DEBUG and ALLOWED_HOSTS
- I know where the main URL list lives

I want to...	Open
Run a command	manage.py
Change a setting	config/settings.py
Add a web address	config/urls.py
Deploy with Gunicorn	config/wsgi.py
Deploy with Uvicorn	config/asgi.py

UP NEXT · STEP 09 **Create an application** python manage.py startapp students